

✓ SESSION 5 — Handling Missing Values in Pandas

Why Handling Missing Values is Important

In real-world datasets, it is very common to encounter missing values.

Examples:

Customer	Age	City
Aman	25	Delhi
Priya	NaN	Mumbai
Rahul	30	Delhi

Missing values can occur due to:

- Data entry errors
- Incomplete forms
- System failures
- Data collection issues

If missing values are not handled properly, they can cause:

- Incorrect analysis
- Errors in machine learning models
- Biased results

Therefore data cleaning is a critical step in data analysis.

Identifying Missing Values

Pandas represents missing values using **NaN**.

Function used:

`isnull()`

Syntax

`df.isnull()`

Purpose

Returns a boolean DataFrame where:

True → value is missing

False → value is present

```
import pandas as pd
import numpy as np

data = {
    "Customer": ["Aman", "Priya", "Rahul", "Sneha"],
    "Age": [25, np.nan, 30, np.nan],
    "City": ["Delhi", "Mumbai", "Delhi", "Bangalore"]
}

df = pd.DataFrame(data)

df
```

	Customer	Age	City
0	Aman	25.0	Delhi
1	Priya	NaN	Mumbai
2	Rahul	30.0	Delhi
3	Sneha	NaN	Bangalore

```
df.isnull()
```

	Customer	Age	City
0	False	False	False
1	False	True	False
2	False	False	False
3	False	True	False

✓ Counting Missing Values

To count missing values per column we use:

```
df.isnull().sum()
```

```
df.isnull().sum()
```

	0
Customer	0
Age	2
City	0
dtype:	int64

✓ Removing Missing Values

Function:

```
dropna()
```

Syntax

```
df.dropna(axis=0, how='any')
```

Parameters

axis

0 → remove rows

1 → remove columns

how

'any' → drop if any value is missing

'all' → drop if all values are missing

```
df.dropna()
```

	Customer	Age	City
0	Aman	25.0	Delhi
2	Rahul	30.0	Delhi

```
import pandas as pd
import numpy as np

data = {
    "Name": ["Alice", "Bob", "Charlie", "David", None],
    "Age": [25, None, 30, None, None],
    "City": ["New York", "London", None, None, None]
}

df = pd.DataFrame(data)

print(df)
```

	Name	Age	City
0	Alice	25.0	New York
1	Bob	NaN	London
2	Charlie	30.0	None
3	David	NaN	None
4	None	NaN	None

```
df.dropna(axis=0)
```

	Name	Age	City
0	Alice	25.0	New York

```
df.dropna(axis=0, how='any')
```

	Name	Age	City
0	Alice	25.0	New York

Rows removed:

Row 1 → Age missing

Row 2 → City missing

Row 3 → Age & City missing

Row 4 → All missing

```
df.dropna(axis=0, how='all')
```

	Name	Age	City
0	Alice	25.0	New York
1	Bob	NaN	London
2	Charlie	30.0	None
3	David	NaN	None

```
df.dropna(axis=1, how='any')
```

0
1
2
3
4

✓ Filling Missing Values

Instead of removing rows, we may replace missing values.

Function:

`fillna()`

Syntax

`df.fillna(value)`

Common strategies:

- Replace with mean
- Replace with median
- Replace with constant value

```
df["Age"].fillna(df["Age"].mean())
```

	Age
0	25.0
1	27.5
2	30.0
3	27.5

dtype: float64

```
# not my code
# FILLING MISSING VALUES:--
import pandas as pd
import numpy as np

data={'customer':['aman','priya','rahul','sneha'],
      'age':[25,np.nan,30,np.nan],
      'city':['delhi','mumbai',np.nan,'banglore']}

df=pd.DataFrame(data)
print(df)
b='dropped'
#df['age'].fillna(b)
c = df['age'].mean()
a = "mehsana"
# df[['age','city']].fillna({df['age'].mean(),df['city'] : a}) # .fillna(value) you can put any values with it
df.fillna({
    "age" : c,
    "city" : "mehsana",
    # "City" : "Ahmedabad"
})
```

```

customer  age    city
0      aman  25.0  delhi
1     priya   NaN  mumbai
2     rahul  30.0   NaN
3     sneha   NaN  banglore

   customer  age    city
0      aman  25.0  delhi
1     priya  27.5  mumbai
2     rahul  30.0  mehsana
3     sneha  27.5  banglore

```

```

value = 44
# df.fillna(value)
# df["Age"].fillna(value)
df.fillna({
    "Name" : "Aman",
    "Age" : value,
    "City" : "Ahmedabad"
})

```

```

   Name  Age    City
0  Alice  25.0  New York
1   Bob  44.0   London
2 Charlie  30.0 Ahmedabad
3  David  44.0 Ahmedabad
4   Aman  44.0 Ahmedabad

```

✓ Replacing Missing Values with Median

Median is often preferred when data contains outliers.

```
df["Age"].fillna(df["Age"].median())
```

```

   Age
0  25.0
1  27.5
2  30.0
3  27.5

dtype: float64

```

✓ Case Study — Cleaning Missing Ages

Goal:

Replace missing ages using average age.

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

```
df
```

	Customer	Age	City
0	Aman	25.0	Delhi
1	Priya	27.5	Mumbai
2	Rahul	30.0	Delhi
3	Sneha	27.5	Bangalore

✓ Handling Missing Values in Categorical Columns

Pandas supports multiple data types in a DataFrame, such as:

- Numerical → Age, Salary
- Categorical → City, Gender, Product
- Boolean → True/False
- Datetime → Dates

Missing values (NaN) can exist in **any type of column**, not just numerical.

Important Clarification

- dropna() works on all columns
- fillna() works on all columns

BUT

The way we **fill missing values depends on the data type**

Handling Numerical vs Categorical Data

Numerical Columns:

- Replace with mean
- Replace with median

Categorical Columns:

- Replace with mode (most frequent value)
- Replace with constant like "Unknown"

Why Mean Does Not Work for Categorical Data

Example:

City column:

Delhi, Mumbai, NaN

We cannot calculate mean of "Delhi" and "Mumbai"

So we use:

- Mode (most frequent value)
- Constant value like "Unknown"

```
import pandas as pd
import numpy as np

data = {
    "Customer": ["Aman", "Priya", "Rahul", "Sneha"],
    "Age": [25, np.nan, 30, np.nan], # Numerical
    "City": ["Delhi", np.nan, "Delhi", "Mumbai"] # Categorical
}

df = pd.DataFrame(data)

print("Original DataFrame:\n")
print(df)
```

Original DataFrame:

	Customer	Age	City
0	Aman	25.0	Delhi
1	Priya	NaN	NaN
2	Rahul	30.0	Delhi
3	Sneha	NaN	Mumbai

✓ 1. Filling Numerical Column (Age)

Use mean or median

```
df["Age"] = df["Age"].fillna(df["Age"].mean())

print(df)
```

	Customer	Age	City
0	Aman	25.0	Delhi
1	Priya	27.5	NaN
2	Rahul	30.0	Delhi
3	Sneha	27.5	Mumbai

✓ 2. Filling Categorical Column Using Mode

Mode = most frequent value

```
# FILLING MISSING VALUES:--
import pandas as pd
import numpy as np

data={'customer':['aman','priya','rahul','sneha'],
      'age':[25,np.nan,30,np.nan],
      'city':['delhi','delhi',np.nan,'banglore']}

df=pd.DataFrame(data)
#print(df)
b='dropped'
#df['age'].fillna(b)
#df['age'].fillna(df['age'].mean()) # .fillna(value) you can put any values with it.
df['city']=df['city'].fillna(df['city'].mode()[0])
```

	customer	age	city
0	aman	25.0	delhi
1	priya	NaN	delhi
2	rahul	30.0	delhi
3	sneha	NaN	banglore

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
df["City"] = df["City"].fillna(df["City"].mode()[0])
print(df)
```

	Customer	Age	City
0	Aman	25.0	Delhi
1	Priya	27.5	Delhi
2	Rahul	30.0	Delhi
3	Sneha	27.5	Mumbai

3. Filling Categorical Column Using Constant

Instead of mode, we can use a constant value.

Example:

"Unknown"

```
df = pd.DataFrame(data)

df["City"] = df["City"].fillna("Unknown")

print(df)
```

	Customer	Age	City
0	Aman	25.0	Delhi
1	Priya	NaN	Unknown
2	Rahul	30.0	Delhi
3	Sneha	NaN	Mumbai

4. Using fillna() for Multiple Columns

We can handle both numerical and categorical columns together.

Example:

- Age → replace with mean
- City → replace with "Unknown"

```
df = pd.DataFrame(data)

df = df.fillna({
    "Age": df["Age"].mean(),
    "City": "Unknown"
})

print(df)
```

	Customer	Age	City
0	Aman	25.0	Delhi
1	Priya	27.5	Unknown

2	Rahul	30.0	Delhi
3	Sneha	27.5	Mumbai

SESSION 6 — Handling Duplicates and Outliers

Two common problems in datasets are:

- Duplicate records
- Outliers (extreme values)

Both can distort analysis results.

Handling Duplicate Rows

Function:

`drop_duplicates()`

Syntax

`df.drop_duplicates()`

Purpose

Removes duplicate rows from the dataset.

Parameters

subset

Specify column(s) to check duplicates

keep

'first' → keep first occurrence

'last' → keep last occurrence

False → remove all duplicates

```
data = {  
    "Customer": ["Aman", "Priya", "Aman", "Rahul"],  
    "Sales": [500, 700, 500, 900]  
}
```

```
df = pd.DataFrame(data)
```

```
df
```

	Customer	Sales
0	Aman	500
1	Priya	700
2	Aman	500
3	Rahul	900

```
df.drop_duplicates()
```

	Customer	Sales
0	Aman	500
1	Priya	700
3	Rahul	900

What are Outliers?

Outliers are extreme values that differ significantly from other observations.

Example:

[10, 12, 11, 13, 500]

Value **500** is an outlier.

Outliers can occur due to:

- measurement errors
- data entry errors
- unusual events

✓ Detecting Outliers using IQR Method

IQR = Interquartile Range

Formula:

$IQR = Q3 - Q1$

Lower Bound = $Q1 - 1.5 \times IQR$

Upper Bound = $Q3 + 1.5 \times IQR$

```
sales = pd.Series([100,120,130,110,115,500])

Q1 = sales.quantile(0.25)
Q3 = sales.quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

outliers = sales[(sales < lower) | (sales > upper)]

outliers
```

```

      0
5  500

dtype: int64
```

✓ Z-Score Concept

Z-score measures how far a value is from the mean.

Formula: